Bahria University, Islamabad

Department of Software Engineering

Artificial Intelligence Lab (Spring-2026)

Teacher: Engr. Saad Mazhar

Student : Talha Naveed

Enrollment : 01-131232-089

Lab Journal: 04

Date: 24th February, 2026

Task No:	Task Wise Marks		Documentation Marks		Total Marks (20)
	Assigned	Obtained	Assigned	Obtained	
1	3		5		
2	3				
3	3				
4	3				
5	3				

Comments:

Signature

Lab No: 04 - Implementing Graph Algorithms

Task 1:

Code

```
import abc
from typing import List
import numpy as np
from collections import deque


class Graph(abc.ABC):
    """
    Base (abstract) Graph class.
    """

    def __init__(self, num_vertices: int, directed: bool = False) -> None:
        """
        Parameters:
            num_vertices (int): number of vertices in the graph
            directed (bool): True for directed graph, False for undirected
        """
        self.num_vertices = num_vertices
        self.directed = directed

    @abc.abstractmethod
    def add_edge(self, v1: int, v2: int, weight: int = 1) -> None:
        """Connects two vertices and assigns weight to the edge."""
        raise NotImplementedError

    @abc.abstractmethod
    def get_adjacent_vertices(self, v: int) -> List[int]:
        """Returns list of neighboring vertices."""
        raise NotImplementedError

    @abc.abstractmethod
    def get_indegree(self, v: int) -> int:
        """Returns indegree of vertex v."""
        raise NotImplementedError

    @abc.abstractmethod
    def get_edge_weight(self, v1: int, v2: int) -> float:
        """Returns weight of edge between v1 and v2."""
        raise NotImplementedError

    @abc.abstractmethod
    def display(self) -> None:
        """Prints the whole graph."""
```

```
        raise NotImplementedError


class AdjacencyMatrixGraph(Graph):
    """
    Graph implementation using an adjacency matrix.
    """

    def __init__(self, num_vertices: int, directed: bool = False) -> None:
        super().__init__(num_vertices, directed)
        self.matrix = np.zeros((num_vertices, num_vertices))

    def add_edge(self, v1: int, v2: int, weight: int = 1) -> None:
        """Connects two vertices."""
        if v1 >= self.num_vertices or v2 >= self.num_vertices or v1 < 0 or v2
< 0:
            raise ValueError(f"Vertices {v1} and {v2} are out of bounds")

        if weight < 1:
            raise ValueError("An edge cannot have weight < 1")

        self.matrix[v1][v2] = weight
        if not self.directed:
            self.matrix[v2][v1] = weight

    def get_adjacent_vertices(self, v: int) -> List[int]:
        if v < 0 or v >= self.num_vertices:
            raise ValueError(f"Cannot access vertex {v}")

        adjacent_vertices: List[int] = []
        for i in range(self.num_vertices):
            if self.matrix[v][i] > 0:
                adjacent_vertices.append(i)
        return adjacent_vertices

    def get_indegree(self, v: int) -> int:
        """Returns indegree of vertex v."""
        if v < 0 or v >= self.num_vertices:
            raise ValueError(f"Cannot access vertex {v}")

        indegree = 0
        for i in range(self.num_vertices):
            if self.matrix[i][v] > 0:
                indegree += 1
        return indegree

    def get_edge_weight(self, v1: int, v2: int) -> float:
        """Returns weight of edge between two vertices."""
        return self.matrix[v1][v2]
```

```python
    def display(self) -> None:
        """Prints the whole graph."""
        for i in range(self.num_vertices):
            for v in self.get_adjacent_vertices(i):
                print(i, "-->", v)

# BFS & DFS Traversal Implementations


def bfs(graph: Graph, start_vertex: int):
    """Implements BFS traversal."""
    visited = []
    queue = deque([start_vertex])
    visited.append(start_vertex)

    print("BFS Traversal: ", end="")
    while queue:

        node = queue.popleft()
        print(node, end=" ")

        for neighbor in graph.get_adjacent_vertices(node):
            if neighbor not in visited:
                visited.append(neighbor)
                queue.append(neighbor)
    print()
    return visited


def dfs(graph: Graph, start_vertex: int):
    """Implements DFS traversal."""
    visited = []

    def dfs_recursive(current_vertex):
        visited.append(current_vertex)
        print(current_vertex, end=" ")

        for neighbor in graph.get_adjacent_vertices(current_vertex):
            if neighbor not in visited:
                dfs_recursive(neighbor)

    print("DFS Traversal: ", end="")
    dfs_recursive(start_vertex)
    print()
    return visited


if __name__ == "__main__":
```

```
g = AdjacencyMatrixGraph(4, directed=True)
g.add_edge(0, 1)
g.add_edge(0, 2)
g.add_edge(1, 2)
g.add_edge(2, 0)
g.add_edge(2, 3)
g.add_edge(3, 3)

print("\n------ Graph Representation ------\n")
print("Graph Edges:")
g.display()
print("-" * 20)

print("\n------ DFS & BFS Traversal ------\n")

bfs(g, 2)  # Expected: 2 0 3 1
dfs(g, 2)  # Expected: 2 0 1 3
print("\n\n")
```

Screenshot

```
(venv) Talhas-MacBook-Air:Lab 04 talha$ python3 Task.py

------ Graph Representation ------

Graph Edges:
0 --> 1
0 --> 2
1 --> 2
2 --> 0
2 --> 3
3 --> 3
--------------------

------ DFS & BFS Traversal ------

BFS Traversal: 2 0 3 1
DFS Traversal: 2 0 1 3
```

Figure 1. Output

Conclusion

In conclusion, this lab provided essential hands-on experience in representing and traversing graphs programmatically. By transitioning from basic dictionary-based graphs to a formal Object-Oriented framework using an Adjacency Matrix, we learned how to structure complex data types robustly in Python.